

## PRÁCTICA 5: ABSTRACCIÓN Y ENCAPSULAMIENTO EN JAVA

### ¿QUÉ BUSCABA LA PRÁCTICA?

#### INTEGRANTE 1 — PONENTE PRINCIPAL

En esta práctica trabajamos dos ideas que parecen muy teóricas, pero que cambian bastante la forma de construir una clase: la abstracción y el encapsulamiento.

Cuando queremos representar algo del mundo real en un programa, no necesitamos guardar absolutamente todo sobre ese objeto, solo lo necesario para resolver el problema.

Y tampoco conviene que cualquier parte del programa pueda cambiar sus datos sin control.

Por eso el objetivo fue diseñar clases en Java eligiendo sus características y funciones, y usando niveles de acceso para proteger la información.

## ABSTRACCIÓN Y ENCAPSULAMIENTO

#### INTEGRANTE 2

Primero está la abstracción. Básicamente consiste en quedarnos con lo importante y dejar fuera detalles que no necesitamos.

Después aparece el encapsulamiento, que busca proteger los datos internos del objeto. Por eso los atributos pueden declararse como privados y el acceso se hace mediante métodos.

Aquí entran los getters, que permiten consultar un valor, y los setters, que permiten modificarlo.

La idea es controlar mejor qué puede verse y qué puede cambiarse.

## UNA DUDA CENTRAL: SETTERS EN CONSTRUCTORES

#### INTEGRANTE 1 — PONENTE PRINCIPAL

Y aquí aparece una duda importante: si un setter ya sabe validar un dato, ¿conviene llamarlo también desde el constructor para no repetir código?

Para responderlo, los cinco integrantes hicimos consultas independientes con inteligencia artificial.

Cada uno revisó un enfoque distinto, como reutilización de código, herencia y polimorfismo, inmutabilidad, responsabilidad de cada método y alternativas de diseño.

Después comparamos los resultados para buscar una postura común.

## ¿QUÉ ENCONTRÓ LA INVESTIGACIÓN?

### INTEGRANTE 5

Lo que se encontró es que usar setters públicos o sobreescribibles dentro de un constructor puede traer problemas.

Por ejemplo, con herencia podría ejecutarse un método de una clase hija antes de que esa parte del objeto termine de inicializarse.

También puede dificultar la creación de objetos inmutables y mezclar dos responsabilidades: construir el objeto y modificarlo después.

Como alternativa, la práctica menciona separar la validación en métodos privados o usar otros mecanismos de creación cuando el caso sea más complejo.

## APLICACIÓN: CLASE EMPLEADO

### INTEGRANTE 1 — PONENTE PRINCIPAL

Con esa idea más clara, pasamos a las clases concretas.

El primer programa fue Empleado. La clase guarda nombre, apellido y salario mensual como atributos privados.

El constructor recibe esos datos y el salario solo se asigna si es mayor que cero.

Después se incluyen getters y setters para consultar o cambiar los valores de forma controlada.

Además, se creó un método que obtiene el salario anual multiplicando el salario mensual por doce.

## RESULTADOS DE EMPLEADO

### INTEGRANTE 3

En MainEmpleado se leen los datos de dos empleados, se crean los objetos y se muestran sus salarios anuales.

Después se aumenta en diez por ciento el salario mensual de cada uno usando los métodos de acceso y modificación.

En la prueba aparecen Juan Perez y Ana Salgado: sus salarios anuales iniciales son 54,000 y 64,800, y después del aumento pasan a 59,400 y 71,280.

Así se comprueba tanto el cálculo como la modificación de los datos.

## CLASE FECHA Y RESULTADOS

### INTEGRANTE 4

El segundo programa fue Fecha.

Aquí los atributos privados son mes, día y año. El constructor recibe los tres valores y se agregan getters y setters para cada uno.

También se creó mostrarFecha(), que imprime la fecha con diagonales.

En la ejecución se ingresó 10/3/2026 y después se modificaron los valores hasta obtener 12/25/2027.

Es un ejemplo sencillo, pero muestra cómo un objeto guarda sus datos y permite cambiarlos mediante sus propios métodos.

## CÓMO SE CONECTA TODO

### INTEGRANTE 1 — PONENTE PRINCIPAL

Entonces, la práctica no se queda en aprender getters y setters por separado.

Primero elegimos qué datos necesita realmente una clase: eso es abstracción.

Luego protegemos esos datos y decidimos cómo se consultan o modifican: ahí entra el encapsulamiento.

Los ejercicios de Empleado y Fecha llevaron esas ideas al código, y la investigación mostró que una decisión aparentemente pequeña, como reutilizar un setter dentro de un constructor, también puede afectar el diseño de la clase.

Al final, sí se cumplieron los objetivos planteados: identificamos características y funciones necesarias, usamos distintos niveles de acceso y aplicamos abstracción y encapsulamiento en Java.

Y más allá de que los programas funcionaran, la idea central fue entender cómo construir clases más claras, con datos protegidos y con una forma controlada de trabajar con ellos.